

Classification project on cardiovascular disease

The data set is taken from Kaggle it has several features including age, height, weight, gender, Systolic blood pressure, diastolic blood pressure, cholesterol, glucose, smoking, alcohol intake, physical activity, and the target variable presence or absence of cardiovascular disease

1. Initial Data Inspection

```
data = pd.read_csv(r"C:\Users\91972\Downloads\health_data.csv\health_data.csv")
df = data.copy()
df.sample(3)
df.info()
df.nunique()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 70000 entries, 0 to 69999
Data columns (total 14 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Unnamed: 0      70000 non-null  int64
1   id              70000 non-null  float64
2   age             70000 non-null  float64
3   gender          70000 non-null  int64
4   height          70000 non-null  float64
5   weight          70000 non-null  float64
6   ap_hi           70000 non-null  float64
7   ap_lo           70000 non-null  float64
8   cholesterol     70000 non-null  int64
9   gluc            70000 non-null  int64
10  smoke           70000 non-null  int64
11  alco            70000 non-null  int64
12  active          70000 non-null  int64
13  cardio          70000 non-null  int64
dtypes: float64(6), int64(8)
memory usage: 7.5 MB
```

Unnamed: 0	70000
id	70000
age	8076
gender	2
height	109
weight	287
ap_hi	153
ap_lo	157
cholesterol	3
gluc	3
smoke	2
alco	2
active	2
cardio	2
dtype: int64	

So we found that there are no null values and we have binary encoded variables.

2. Removal of Unnecessary Features and making features consistent

Now removing unnecessary features like unnamed, ID `df = df.drop(['Unnamed: 0', 'id'], axis=1)`

Since the age column is given in days it converting it into year and using the height and weight feature to get the BMI feature which is equal to weight divided by height square and then dropping height and weight

```
df['age'] = round(df['age']/365)
df['height'] = df['height']/100
df['BMI'] = df['weight']/((df['height'])**2)
```

Checking whether the data set is balanced or imbalance by checking the proportion of target variable that is a cardiovascular disease

```
print(df['cardio'].value_counts()/df['cardio'].count())  
print('Dataset is balanced')
```

```
cardio  
0    0.5003  
1    0.4997  
Name: count, dtype: float64  
Dataset is balanced
```

Since it is a medical data set blood pressure like systolic blood pressure or diastolic blood pressure has a certain range beyond that is it either the typo or error false values so we will treat them either removing them either doing resolve of typo and etc

Conditions

```
# Investigation ap_hi , ap_lo features  
# Systolic blood pressure (ap_hi)  
  
# Range: 90 - 200 mmHg  
  
# Diastolic blood pressure (ap_lo)  
  
# Range: 60 - 130 mmHg  
  
# Mandatory condition  
# ap_lo < ap_hi
```

- a. So first we will check that whether condition is fulfilled the condition is diastolic blood pressure is always less than or equal to systolic blood pressure given that diastolic blood pressure is greater or equal to 1000 so we have found that there is a typo of extra zero So what we do we change that zero by dividing by 10

```
df[(df['ap_lo'] >= df['ap_hi']) & (df['ap_lo'] >= 1000)] = df[(df['ap_lo'] >= df['ap_hi']) & (df['ap_lo'] >= 1000)]/10
```

- b. now we have checked whether the any of the blood pressure is equal to zero as because there is no such case where the blood pressure can be zero so it just a mistake so we have removed all the rows where blood pressure is zero

```
df[(df['ap_lo']==0)|(df['ap_hi']==0)]  
  
df = df[((df['ap_lo']!=0) & (df['ap_hi']!=0))]
```

- c. Now we have checked whether the blood pressure is negative that can never be possible so if it is it is either the typo or an error value so on observing the data set we have found that it is a type O So we have converted that negative value into positive

```
print(df[(df['ap_hi']<0)|(df['ap_lo']<0)].to_string())
print("""Except the row with index 60106 all the other roads are reliable
      we just need to convert the given value of systolic blood pressure into positive
      so which makes the row rel and we'll remove that particular row index 60106""")

# 1- convert vaues into positive then remove row 60106
df = df.drop(index=[60106])

idx_neg_bp = list(df[(df['ap_hi']<0)|(df['ap_lo']<0)].index)
df.loc[idx_neg_bp, 'ap_hi'] = - df.iloc[idx_neg_bp, 2]
```

- d. In this step we have checked that each of the blood pressure either any of them for example systolic blood pressure or diastolic blood pressure is greater than 1,00,000 if it is then we observe whether is a typo or not then we have checked whether they are greater than 10000 then whether they are greater than 1000 if then is it a typo or error if a typo then we have managed it otherwise delete

```
# Checking Large values of BP

df[(df['ap_hi']>100000) | (df['ap_lo']>100000)]      #no rows for this condit

df[(df['ap_hi']>10000) | (df['ap_lo']>10000)]      # Converting ap_hi
idx1=df[(df['ap_hi']>10000) | (df['ap_lo']>10000)].index
df.loc[idx1, 'ap_hi'] = df.loc[idx1, 'ap_hi']/100

df[(df['ap_hi']>1000) | (df['ap_lo']>1000)]      # Converting ap_hi
idx2=df[(df['ap_hi']>1000) | (df['ap_lo']>1000)].index
df.loc[idx2, 'ap_hi'] = df.loc[idx2, 'ap_hi']/10

df[(df['ap_hi']>1000) | (df['ap_lo']>1000)]      # Converting ap_hi
df=df.drop(index=df[(df['ap_hi']>1000) | (df['ap_lo']>1000)].index)
```

- e. And at last we have checked since it is a blood pressure so it have a certain range beyond that range it will be a error so we have checked externa sources and found that this range must be less than 300 by so if it is greater than 300 then we will delete it

```
df[(df['ap_hi']>300) | (df['ap_lo']> 300)]
df = df.drop(index=df[(df['ap_hi']>300) | (df['ap_lo']>300)].index )

df.describe()
```

	ap_hi	ap_lo	BMI_kg/m2
count	69906.000000	69913.000000	69913.000000
mean	124.908024	81.629126	27.209890
std	21.554698	10.078015	6.641613
min	0.100000	1.000000	0.367309
25%	120.000000	80.000000	23.795360
50%	120.000000	80.000000	26.259585
75%	140.000000	90.000000	30.110279
max	240.000000	208.800000	298.666667

On observing that DF dot describe we found that the blood pressure features are now OK but there are several mistakes in the body mass index so by the external sources I have found that it must be between 10 to 100 because it is very rare it to be beyond 70 but we will assume it at below 100 is it normal So beyond that we are removing

```
# NOW checking BMI a, its typical value must be between 10 to 100
df[(df['BMI_kg/m2'] <10) | (df['BMI_kg/m2'] >100) ]
# Removing
df = df.drop(index=df[(df['BMI_kg/m2'] <10) | (df['BMI_kg/m2'] >100)].index)
```

Although BMI was examined before height and weight, the same physiological validation logic was applied. This ordering does not affect correctness; it reflects exploratory analysis sequence.

Note

In the data set it is not specified whether the encoded gender feature has the value one means or the with zero equal to females so we have used some Observation by calculating the mean height of both the encoders 0 and one and we have found that the mean height of the value encoded zero is 161cm while the other which is encoded by one is 169 so it is in general or naturally we have seen that males are taller than females so we have assumed that the encoder one equals male and the encoder 0 equal to female

```
print(data.groupby('gender')[['height','weight']].mean() )
print('1 - Male,0- Female')
```

✓ 0.0s

	height	weight
gender		
0	161.355612	72.565605
1	169.947895	77.257307
1 - Male,0- Female		

One last we have Nans in ap_hi

```
df = df.drop_duplicates()
# df.info()
df['ap_hi'].sort_values(ascending=False)
df.isna().sum()
```

✓ 0.0s

```
age_yr      0
gender      0
ap_hi       7
ap_lo       0
cholesterol  0
gluc        0
smoke       0
alco        0
active      0
cardio      0
BMI_kg/m2   0
dtype: int64
```

Since these are very few – 7 missing values at random – removing them

```
df[df['ap_hi'].isna()] # These are very few 7 missing values at random --- removing them
df = df.dropna()
```

Machine Learning Modeling and Model Selection

Objective : Built an ML Model to predict whether a patient based on some inputs has cardiovascular disease or not .

Train–Test Strategy After completing data preprocessing and feature engineering, the dataset was divided into training and testing subsets using **stratified train–test splitting**, ensuring that the proportion of the target classes (cardiovascular disease: present/absent) remained consistent across both sets.

Stratification was used to avoid sampling bias and to ensure fair evaluation of model performance.

All models were primarily evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- Cross-validation performance

Baseline Models

Logistic Regression

Logistic Regression was implemented using **GridSearchCV** for hyperparameter tuning.

- Accuracy: ~72%
- Performance remained stable across folds

This model served as a linear baseline and established the minimum achievable performance using linear decision boundaries.

Polynomial Logistic Regression

Polynomial feature expansion was introduced to capture nonlinear relationships.

- Accuracy: ~72%

The introduction of polynomial interactions did not significantly improve performance, suggesting limited nonlinear interaction effects in the feature space detectable by linear models.

Tree-Based Models

Decision Tree Classifier

A Decision Tree was trained using GridSearchCV.

- Accuracy: ~73%
- Precision: ~71%
- Recall: ~78%
- F1-score: 70–74%

The model showed slightly improved recall but exhibited higher variance compared to logistic regression.

Random Forest Classifier

Random Forest was tuned using GridSearchCV.

- Accuracy: ~74–76%
- Precision: ~77%
- Recall: ~70%

Random Forest demonstrated improved generalization performance compared to single decision trees, benefiting from bagging and feature randomness.

Probabilistic Model

Categorical Naïve Bayes

- Accuracy: ~72%
- Precision: ~76%

The model performed comparably to logistic regression but did not provide substantial improvement.

Support Vector Machine

An SVM classifier initially showed:

- Accuracy: ~82% (single split evaluation)

However, after applying cross-validation:

- Precision: ~72%

Additionally, training time was significantly high (~25 minutes per fit), making extensive hyperparameter tuning computationally expensive.

The cross-validation results indicated that the initial 82% performance was likely due to variance from a single train–test split rather than true generalization improvement.

Gradient Boosting

Gradient Boosting achieved:

- Accuracy: ~79% (training/test split)
- Precision: 77–81%

However, cross-validation performance dropped to ~72%, indicating variance sensitivity and potential overfitting when model complexity increased.

K-Nearest Neighbors

- Accuracy: ~73%
- Precision: ~76%
- Recall: ~70%

KNN showed moderate performance but did not outperform ensemble methods.

Bagging and Voting Classifiers

Bagging (with SVM base estimator)

- Accuracy: ~78%

Voting Classifier

- Accuracy: ~73%

Ensemble aggregation did not significantly improve cross-validated performance beyond Random Forest.

PCA + SVM

Dimensionality reduction using PCA followed by SVM classification resulted in:

- Accuracy: ~72%

This indicates that dimensionality reduction did not enhance separability in this dataset.

Feature Transformation Experiment: Blood Pressure Discretization

To evaluate whether threshold-based binning improves performance, systolic and diastolic blood pressure values were discretized into categorical ranges based on clinical intervals.

Two datasets were created:

1. Original continuous BP features
2. Discretized BP features

Blood Pressure Discretization Strategy

To evaluate whether clinically meaningful categorization improves predictive performance, systolic (ap_hi) and diastolic (ap_lo) blood pressure values were discretized into ordinal categories based on standard hypertension guidelines.

1. Systolic Blood Pressure (ap_hi)

Systolic Range (mmHg)	Clinical Category	Encoded Value
< 120	Normal	0
120–129	Elevated	1
130–139	Hypertension Stage 1	2
140–179	Hypertension Stage 2	3
≥ 180	Hypertensive Crisis	4

2. Diastolic Blood Pressure (ap_lo)

Diastolic Range (mmHg)	Clinical Category	Encoded Value
< 80	Normal	0
80–89	Hypertension Stage 1	2
90–119	Hypertension Stage 2	3
≥ 120	Hypertensive Crisis	4

Additionally:

- If systolic is 120–129 mmHg **and** diastolic < 80 mmHg → categorized as **Elevated (1)**.

In Short

The continuous systolic and diastolic blood pressure values were transformed into ordinal categories using clinically defined hypertension thresholds. Each range was encoded numerically from 0 (Normal) to 4 (Hypertensive Crisis). This transformation was intended to assess whether clinically meaningful binning improves predictive separability compared to continuous measurements.

NOW

The final three models (Random Forest, Gradient Boosting, XGBoost) were evaluated on both datasets using cross-validation.

Cross-Validated Precision Comparison

Model	Original Data	Discretized Data
Random Forest	75.73%	75.58%
Gradient Boosting	75.21%	75.18%
XGBoost	73.61%	74.01%

The negligible differences indicate that discretization did not significantly impact predictive performance. Tree-based models inherently learn optimal split thresholds, making manual binning unnecessary.

6. Final Model Selection

Across all experiments:

- Linear models plateaued around ~72%.
- Ensemble methods (RF, GB) improved performance to ~75–78%.
- Increasing model complexity beyond this did not produce substantial gains.

This convergence suggests that performance is likely limited by feature-set predictive capacity and inherent data noise rather than model choice.

Random Forest was selected as the final model due to:

- Slightly superior and stable cross-validated performance
- Lower variance compared to boosting
- Computational efficiency
- Better interpretability through feature importance

Final Model Random Forest with original dataset

Final Model-

```
rf1 = RandomForestClassifier(max_depth=10, max_features=4, min_samples_leaf=45,  
                             min_samples_split=30, n_estimators=200,  
                             random_state=45, oob_score=True)  
  
rf1.fit(X_train, y_train)
```

Final Dataset-

```
X_train.sample(5)
```

✓ 0.0s

	age_yr	gender	ap_hi	ap_lo	cholesterol	gluc	smoke	alco	active	BMI_kg/m2
10615	64.0	0.0	110.0	70.0	0.0	0.0	0.0	0.0	1.0	25.970116
66295	48.0	1.0	120.0	80.0	0.0	0.0	0.0	0.0	0.0	25.180785
30432	58.0	0.0	120.0	80.0	0.0	0.0	0.0	0.0	1.0	29.744200
9068	48.0	1.0	120.0	80.0	0.0	0.0	0.0	0.0	1.0	28.719723
25207	48.0	0.0	140.0	90.0	0.0	0.0	0.0	0.0	1.0	32.882414

```
print(rf1.oob_score_)  
fimp = rf1.feature_importances_*100  
print(fimp)  
inp_f = rf1.feature_names_in_  
print(inp_f)
```

✓ 0.0s

```
0.731242206235012  
[12.15586703  0.36787897 51.56064224 19.45425862  9.83991242  0.83072254  
 0.383157    0.14611217  0.55338851  4.7080605 ]  
['age_yr' 'gender' 'ap_hi' 'ap_lo' 'cholesterol' 'gluc' 'smoke' 'alco'  
 'active' 'BMI_kg/m2']
```

Random Forest was selected as the final model due to:

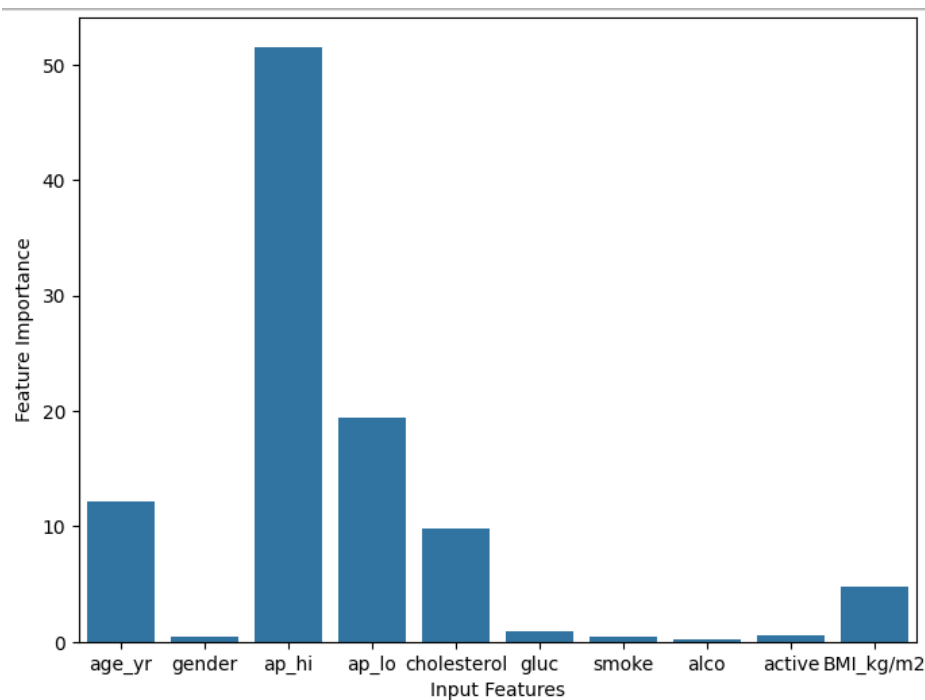
- Slightly superior and stable cross-validated performance (~75–76%)
- Lower variance compared to boosting-based models
- Computational efficiency relative to SVM and boosting methods
- Better interpretability through feature importance analysis
- Consistent generalization confirmed by Out-of-Bag (OOB) score (~73%)

Model Stability and Generalization

- Cross-validation performance (~75–76%) and OOB score (~73%) were closely aligned.
- The small gap between training performance and validation metrics indicates controlled variance.
- No evidence of severe overfitting or underfitting.
- Performance consistency across validation strategies supports model reliability

Feature Importance Insights

- **Systolic blood pressure (ap_hi)** contributed over 50% of total feature importance.
- **Diastolic blood pressure (ap_lo)** and **age** were the next most influential predictors.
- Combined blood pressure features contributed approximately 70% of model decision-making.
- Cholesterol showed moderate importance.
- Lifestyle-related features (smoking, alcohol consumption, physical activity) had minimal impact.



Performance Plateau Observation

- Multiple nonlinear models (Random Forest, Gradient Boosting, XGBoost) converged within a narrow performance range (~73–78%).
- Increasing model complexity did not produce substantial gains.
- This suggests performance is likely limited by:
 - Feature-set predictive capacity
 - Presence of irreducible noise in the dataset
- Further improvement would likely require richer or more informative features rather than more complex algorithms.

Interpreting the Model Final result (Test Data)

```
# Last Model testing on test data
print(classification_report(y_test,rf1.predict(X_test)))
```

✓ 0.5s

	precision	recall	f1-score	support
0.0	0.71	0.76	0.73	6400
1.0	0.75	0.69	0.72	6632
accuracy			0.73	13032
macro avg	0.73	0.73	0.73	13032
weighted avg	0.73	0.73	0.73	13032

0 = No Disease , 1= Disease

Intrepret how good this model predicts:

Main motto –

1. To predict correctly if individual has disease(class 1)
2. Minimize false negatives (maximize recall for disease class). i.e. To avoid guessing no disease even the individual have disease because it will harm the patient in future.

Class-wise Performance

Class 1 (Disease Present)

- Precision: 0.75 – Out of all patients predicted as having disease, 75% truly have the disease.
- Recall: 0.69 – Out of all patients who actually have the disease, 69% were correctly identified by the model.
- F1-score: 0.72

Class 0 (No Disease)

- Precision: 0.71
- Recall: 0.76
- F1-score: 0.73

The model correctly identifies most healthy patients.

NOTE:

- When the model predicts disease, it is correct 75% of the time.
- However, it misses about 31% of actual disease cases (since recall = 69%).

This is the critical part.

Important Medical Insight

In a cardiovascular prediction setting:

False Negatives (missed disease cases) are more dangerous than False Positives.

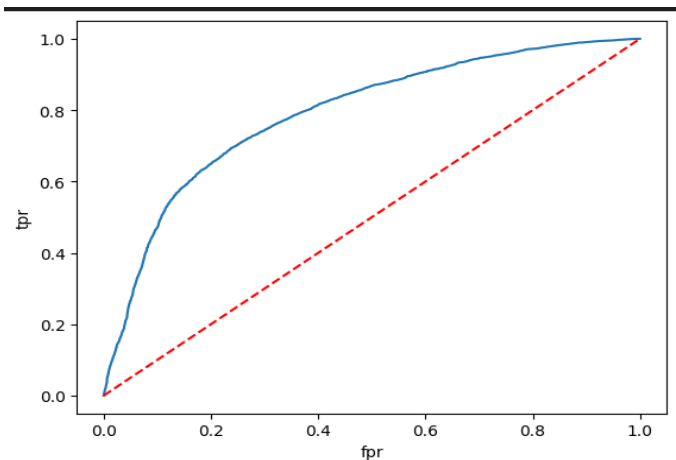
ROC-AUC Curve

```
# ROC- AUC curve
from sklearn.metrics import roc_curve
fpr,tpr,thresholds = roc_curve(y_test,y_scores)

✓ 0.0s

sns.lineplot(x=fpr,y=tpr)
plt.plot([0,1],[0,1],linestyle='--',color='red')
plt.xlabel('fpr')
plt.ylabel('tpr')

✓ 42.7s
```



By the Youdan's J statistic- the optimal threshold is = 0.5 (ie same as model's default threshold)

```
diff = (tpr-fpr)
best_thresh = diff.argmax()
best_t = thresholds[best_thresh]
best_t

✓ 0.0s

np.float64(0.49984802105795084)
```

Note:

In Disease prediction and in the case of Cardiovascular Disease which is a very harmful disease, we might diagnose a patient if he/she has a chance of approximately 30 % or 40 % of being affected by this disease.

So , on checking precision on three different thresholds 0.3, 0.35, 0.4

On threshold >= 0.3

```
# threshold 0.3
y_pred_03 = (rf1.predict_proba(X_test)[:,-1] > 0.3).astype(int)
print(classification_report(y_test, y_pred_03))

✓ 0.4s
```

	precision	recall	f1-score	support
0.0	0.79	0.44	0.57	6400
1.0	0.62	0.89	0.73	6632
accuracy			0.67	13032
macro avg	0.71	0.67	0.65	13032
weighted avg	0.71	0.67	0.65	13032

Recall = 0.89 – out of all patients with disease model correctly identify 89 % of them

Precision = 0.62 – out of all disease predictions only 62 % of them actually was affected

On threshold >= 0.35

```
# threshold 0.35
y_pred_035 = (rf1.predict_proba(X_test)[:,-1] > 0.35).astype(int)
print(classification_report(y_test, y_pred_035))
```

✓ 0.4s

	precision	recall	f1-score	support
0.0	0.77	0.54	0.64	6400
1.0	0.66	0.85	0.74	6632
accuracy			0.70	13032
macro avg	0.71	0.69	0.69	13032
weighted avg	0.71	0.70	0.69	13032

Recall =0.85 – out of all patients with disease model correctly identify 85 % of them

Precision = 0.66 – out of all disease predictions only 66 % of them actually was affected

On threshold >= 0.4

```
# threshold 0.4
y_pred_04 = (rf1.predict_proba(X_test)[:,-1] > 0.4).astype(int)
print(classification_report(y_test, y_pred_04))
```

✓ 0.3s

	precision	recall	f1-score	support
0.0	0.75	0.63	0.68	6400
1.0	0.69	0.80	0.74	6632
accuracy			0.71	13032
macro avg	0.72	0.71	0.71	13032
weighted avg	0.72	0.71	0.71	13032

Recall =0.80 – out of all patients with disease model correctly identify 80 % of them

Precision = 0.69 – out of all disease predictions only 69 % of them actually was affected

Threshold	Precision (%)	Recall (%)
0.3	62	89
0.35	66	85
0.4	69	80

As the decision threshold increases, precision improves while recall decreases, illustrating the inherent tradeoff between false positives and false negatives.

The optimal threshold will be decided in accordance with several factors –

- **Population health impact** – In screening programs, higher recall is prioritized to minimize missed disease cases.
- **Diagnostic cost and resource constraints** – Higher false positives may increase follow-up testing burden.
- **Risk tolerance** – In high-risk cardiovascular settings, missing true positives may carry greater consequences than over-diagnosis.

The optimal threshold depends on the operational objective. For screening purposes, a lower threshold (0.30–0.35) may be preferred to maximize recall, whereas in resource-constrained clinical settings, a higher threshold (e.g., 0.40) may be selected to reduce false positives.

Overall Conclusion

This study developed and evaluated multiple machine learning models for cardiovascular disease prediction using a dataset of 70,000 patient records. After data preprocessing, feature engineering (including BMI creation and blood pressure handling), and stratified train–test splitting, several classification algorithms were systematically compared using cross-validation.

Linear models achieved baseline performance (~72%), while ensemble methods such as Random Forest, Gradient Boosting, and XGBoost improved predictive accuracy to approximately 75–78%. However, performance across strong nonlinear models converged within a narrow range, suggesting that further gains were limited by the predictive capacity of the available features rather than model complexity.

Random Forest was selected as the final model due to:

- Stable cross-validated performance (~75–76%)
- Consistent Out-of-Bag (OOB) score (~73%)
- Controlled variance
- Computational efficiency
- Clear interpretability through feature importance analysis

Feature importance revealed that systolic and diastolic blood pressure were the dominant predictors, contributing the majority of the model’s decision-making influence. Age and cholesterol showed moderate impact, while lifestyle-related variables had relatively low contribution.

Threshold analysis demonstrated the trade-off between precision and recall. Lowering the decision threshold increased recall for the disease class, reducing missed cases at the expense of higher false positives. The optimal threshold therefore depends on the intended clinical objective (e.g., screening vs. diagnostic triage).

Overall, the model demonstrates moderate but stable predictive performance and provides meaningful insights into cardiovascular risk factors within the dataset.

Limitations

Despite structured experimentation and validation, several limitations should be acknowledged:

1. Feature Set Limitation

The dataset primarily includes demographic and basic clinical indicators. The absence of detailed biomarkers, genetic data, imaging results, or longitudinal history likely constrains predictive performance.

2. Irreducible Noise

Cardiovascular disease is influenced by complex physiological and genetic factors. Some variability in outcomes cannot be captured by the available features, limiting achievable accuracy.

3. Single Dataset Evaluation

The model was trained and evaluated on a single dataset. External validation on independent populations would be required to assess generalizability.

4. **Threshold Context Dependence**

The optimal decision threshold varies depending on whether the application is population screening or clinical triage. A universal threshold may not be appropriate.

5. **Class Balance in Dataset**

The dataset is relatively balanced. In real-world screening scenarios, disease prevalence may be significantly lower, which could impact precision and overall model behavior.

6. **Interpretability Constraints**

Although feature importance was analyzed, tree-based importance measures may be biased toward continuous variables. More advanced explainability methods (e.g., SHAP) could provide deeper insight.